INSTITUT DE FORMATION ET DE RECHERCHE EN INFORMATIQUE

IFRI — Cotonou

CAMPUSDOCS

Système de Gestion des Actes Administratifs

DOSSIER DE MODÉLISATION UML

Diagrammes de conception et d'architecture

Technologie : Java 17 · JavaFX · Spring Boot 3.2 · MySQL 8

Année académique : 2025–2026

Table des matières

1. Introduction

CampusDocs est un système de gestion des actes administratifs développé pour l'IFRI (Institut de Formation et de Recherche en Informatique) de Cotonou. Il permet aux étudiants (Usagers) de soumettre des demandes de documents administratifs — attestations d'inscription, bulletins de notes, relevés de notes — et aux agents administratifs de traiter ces demandes, générer les documents PDF correspondants et les mettre à disposition.

Ce dossier présente l'ensemble des diagrammes UML produits lors de la phase de modélisation du système, organisés en deux grandes catégories :

- Diagrammes comportementaux : cas d'utilisation, séquences, activités
- Diagrammes structurels : composants, déploiement

L'architecture technique retenue est une architecture client-serveur à trois couches : un client riche JavaFX, une API REST Spring Boot sécurisée par JWT, et une base de données MySQL.

2. Diagramme de cas d'utilisation

Les diagrammes de cas d'utilisation modélisent les fonctionnalités principales du système du point de vue des acteurs externes. Chaque diagramme illustre les interactions entre les acteurs et le système, en mettant en évidence les différents cas d'utilisation, leurs relations (include, extend) et les objectifs que les utilisateurs peuvent atteindre.

Description du diagramme de cas d'utilisation

Ce diagramme représente le fonctionnement global d'un système de gestion des actes administratifs en mettant en évidence les différents acteurs ainsi que leurs interactions avec le système.

Le système fait intervenir trois acteurs principaux :

- **Administrateur**
- **Agent administratif**
- **Usager**

1. Administrateur

L'administrateur dispose des droits les plus élevés et intervient dans la gestion globale du système. Il peut :

- Gérer les comptes utilisateurs, ce qui inclut :
    - Ajouter un agent administratif
    - Supprimer un utilisateur
    - Consulter les informations d'un utilisateur
    - Modifier les informations d'un utilisateur
- Superviser les opérations
- Configurer le système

2. Agent administratif

L'agent administratif est chargé du traitement des demandes. Il peut :

- Enregistrer une demande
- Traiter les dossiers, avec plusieurs actions possibles :
    - Valider une demande
    - Rejeter une demande
    - Vérifier les informations (incluse dans le traitement)
- Consulter les informations

- Délivrer les actes administratifs

3. Usager

L'usager est le bénéficiaire du service. Il peut :

- Soumettre une demande
- Consulter l'état de sa demande
- Récupérer un acte administratif délivré
- Annuler une demande d'acte administratif

4. Authentification

Toutes les actions effectuées par les acteurs nécessitent une étape préalable :
S'authentifier

Cela est représenté par la relation « include », indiquant que l'authentification est obligatoire avant toute interaction avec le système.

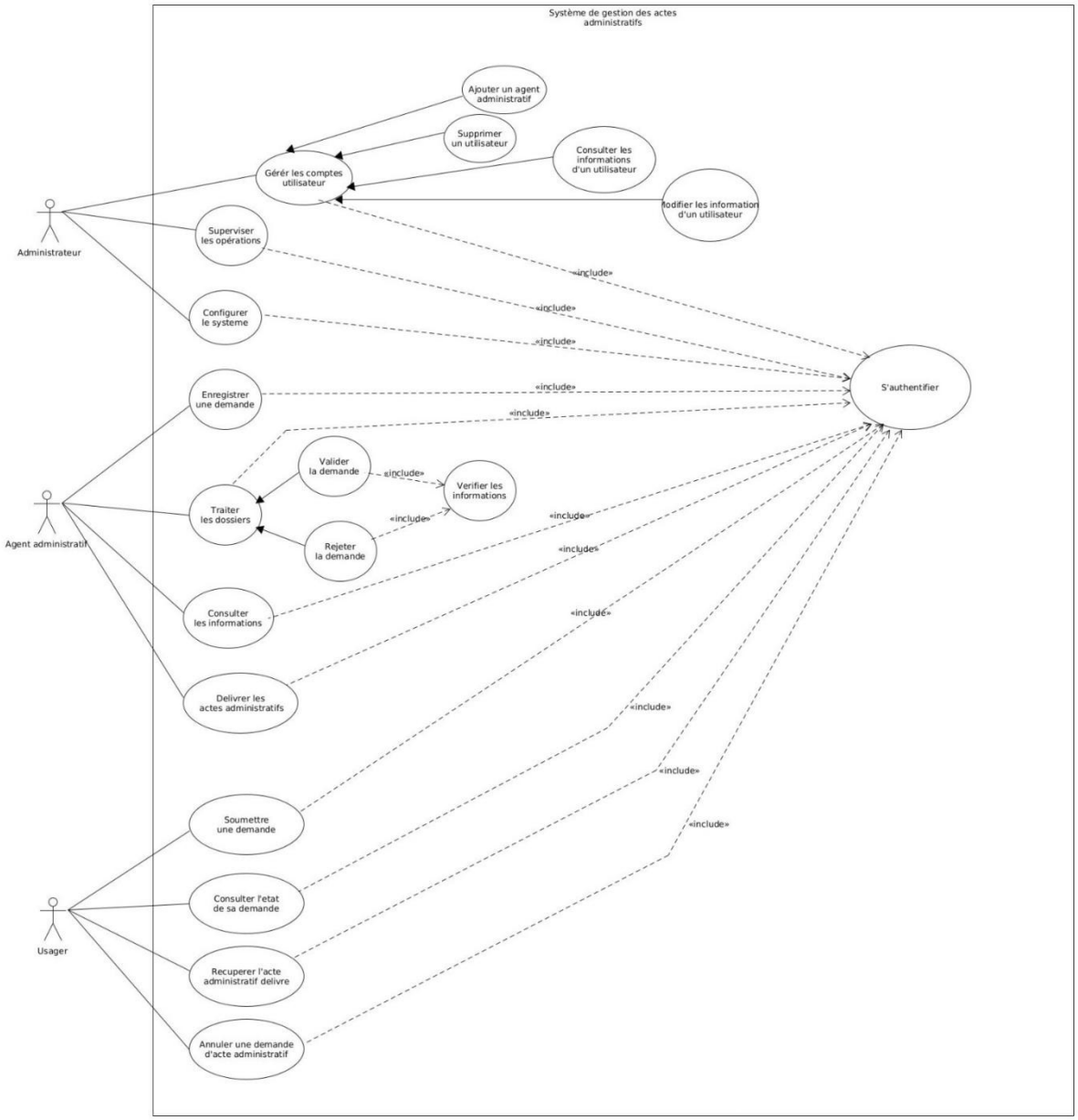

Figure 1 — Diagramme de cas d'utilisation

2. Diagramme de classe

Les diagrammes de classes modélisent la structure statique du système en représentant les différentes classes, leurs attributs, leurs méthodes ainsi que les relations qui les lient (associations, héritage, dépendances). Chaque diagramme illustre l'organisation interne du système et la manière dont les entités interagissent sur le plan structurel.

Description du diagramme de classe

1. Ce diagramme de classes représente la structure statique du système de gestion des actes administratifs. Il met en évidence les différentes classes, leurs attributs, leurs méthodes ainsi que les relations entre elles.

1. Classe Utilisateur (classe mère)

La classe Utilisateur est une classe centrale qui contient les informations communes à tous les types d'utilisateurs :

- Attributs : id, nom, prénom, email, mot de passe, statut (actif)
- Méthode : s'authentifier()

Elle sert de classe parent pour les autres types d'utilisateurs.

2. Classes dérivées

Le système distingue trois types d'utilisateurs :

Usager

- Attributs : matricule, filière, niveau
- Méthode : faireDemande()
  👉 Il représente l'étudiant ou le demandeur d'actes administratifs.

Agent administratif

- Attribut : service
- Méthodes :
  - traiterDemande()
  - générerRapport()
  - délivrerActe()

👉 Il est chargé du traitement des demandes.

Administrateur

- Méthodes :
    - créerCompte()
    - supprimerCompte()
    - exporterDonnées()

👉 Il gère les comptes et le système.

3. Classe Demande

La classe Demande représente une requête effectuée par un usager.

- Attributs : id, type, date, statut, date de traitement, commentaire
- Méthodes :
    - soumettre()
    - valider()
    - rejeter()
    - annuler()
    - consulterStatut()
    - générerActe()

👉 Un usager peut faire plusieurs demandes (0..n).

4. Classe Pièce justificative

Elle représente les documents fournis pour appuyer une demande.

- Attributs : id, type, nomFichier, cheminFichier, dateUpload

👉 Une demande peut contenir plusieurs pièces justificatives (0..n).

5. Classe Acte administratif

Cette classe représente le document final délivré.

- Attributs : id, type, numéro, date de création, statut d’envoi
- Méthodes :
    - générer()
    - envoyer()
    - télécharger()
    - enregistrer()

👉 Une demande peut générer un ou plusieurs actes administratifs.

6. Spécialisation des actes

La classe ActeAdministratif est généralisée en plusieurs types :

- Attestation de scolarité
- Attestation de réussite
- Relevé de notes
- Certificat de scolarité
- Autorisation d'absence
- Diplôme

Cela montre une relation d'héritage (généralisation).

---

7. Relations entre les classes

- Usager → Demande : un usager peut faire plusieurs demandes
- Demande → Pièce justificative : une demande contient plusieurs pièces
- Agent administratif → Demande : il traite les demandes
- Agent administratif → Acte administratif : il génère et envoie les actes
- Administrateur → Utilisateur : il gère les comptes
- Pièce justificative → Agent administratif : les pièces sont vérifiées

窗体顶端

窗体底端

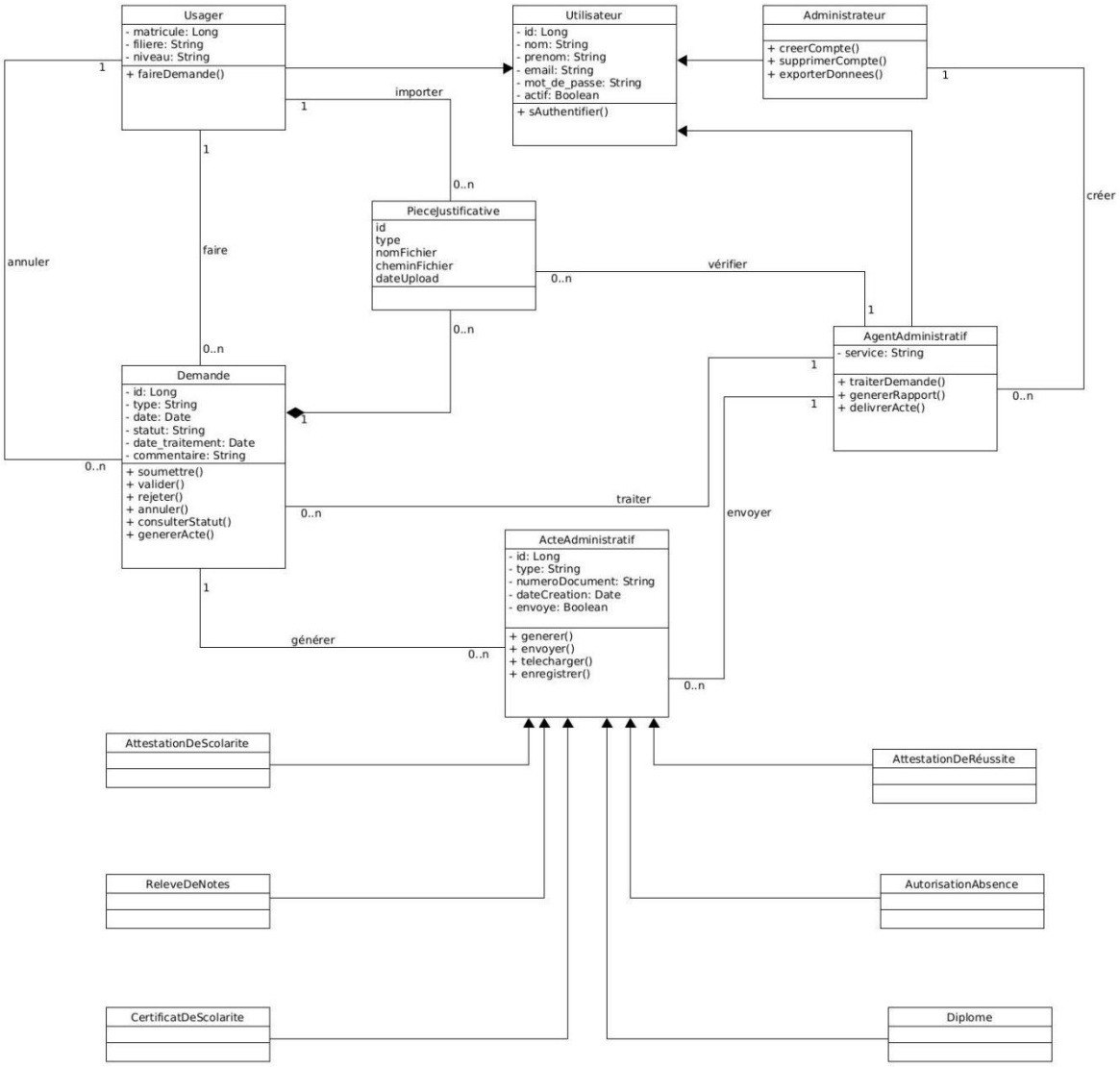

Figure 2— Diagramme de classe

3. Diagrammes de Séquence

Les diagrammes de séquence modélisent les interactions chronologiques entre les acteurs et les composants du système pour chaque cas d'utilisation principal. Chaque diagramme illustre le flux de messages échangés, les conditions alternatives (fragments alt) et les retours du système.

3.1 S'authentifier

> *Acteurs : Utilisateur (Usager, Agent ou Admin). Précondition : l'utilisateur dispose d'un compte actif.*

Ce diagramme décrit le processus d'authentification d'un utilisateur dans CampusDocs. L'utilisateur saisit ses identifiants (email + mot de passe) sur la PageAuth, qui transmet la requête au Système. Ce dernier recherche l'utilisateur en base, vérifie le mot de passe hashé via PBKDF2/SHA-256, puis génère un token JWT en cas de succès.

- En cas de succès (ok == true) : le token JWT est renvoyé, le SessionManager est alimenté et l'utilisateur est redirigé vers son tableau de bord selon son rôle (Admin, Agent, Étudiant).
- En cas d'échec (ok == false) : un message ConnexionEchouee est retourné et l'utilisateur peut réessayer.

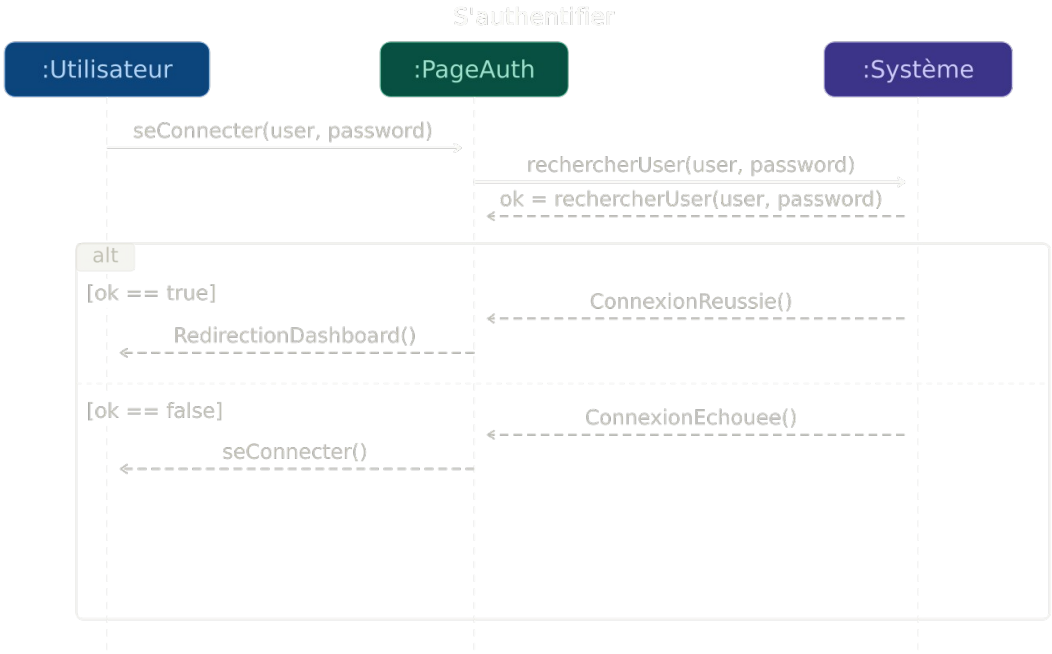

Figure 1 — Diagramme de séquence : S'authentifier

3.2 Créer un compte Agent

> *Acteurs : SuperAdmin (Administrateur). Précondition : l'administrateur est authentifié avec le rôle ADMIN.*

Ce diagramme décrit la procédure de création d'un compte agent administratif par un SuperAdmin. Après authentification, l'administrateur accède au DashboardAdmin, remplit les informations de l'agent (nom, prénom, email, service, mot de passe) et soumet le formulaire.

- En cas de données valides (ok == true) : le système enregistre l'agent avec le mot de passe hashé (PBKDF2 + sel), la date de création est automatiquement renseignée, et un accusé de réception est affiché.
- En cas d'informations invalides (ok == false) : le système retourne un message infosInvalides et propose de réessayer.
- En cas d'échec d'authentification initiale : le flux est interrompu et un message AuthImpossible est émis.

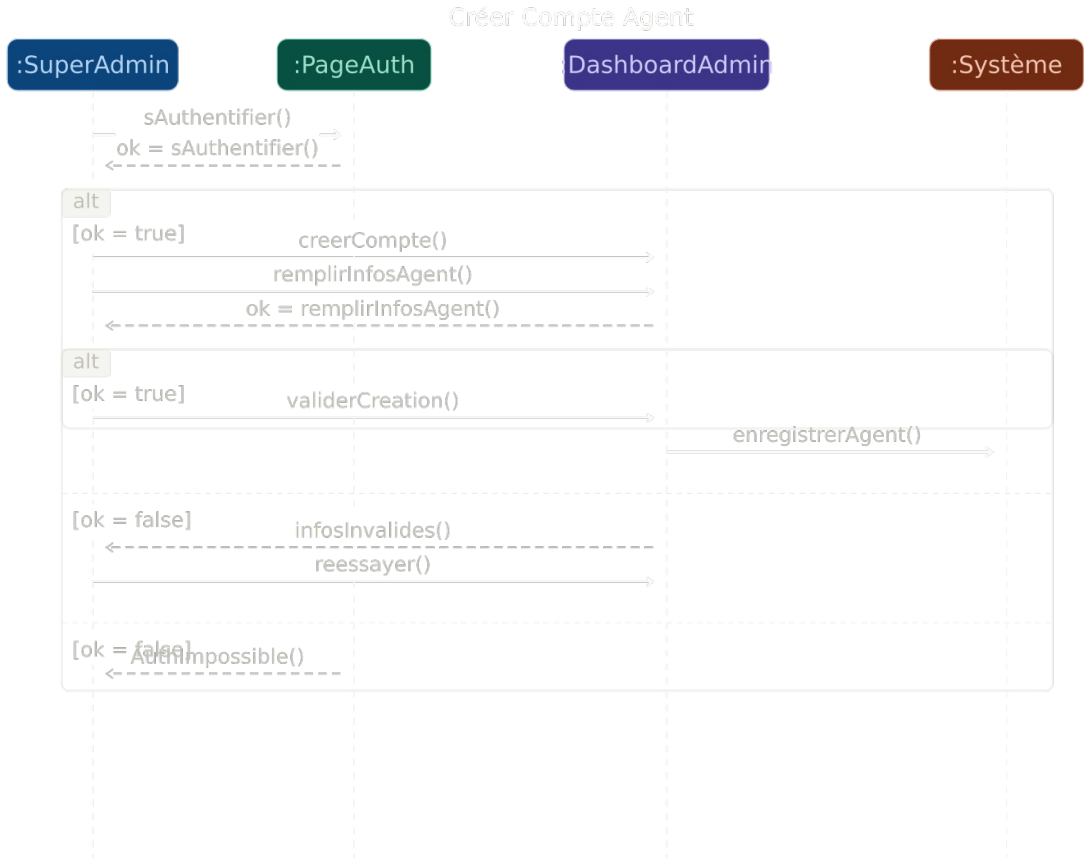

Figure 2 — Diagramme de séquence : Créer un compte Agent

3.3 Faire une demande

> *Acteurs : Utilisateur (Usager/Étudiant). Précondition : l'utilisateur est authentifié.*

Ce diagramme modélise le processus de soumission d'une demande d'acte administratif. Une fois authentifié, l'utilisateur accède à la PageDemande, choisit le type de document souhaité (attestation, bulletin, relevé), complète les informations requises et joint les pièces justificatives nécessaires.

- En cas de pièces valides (ok == true) : la demande est enregistrée dans le système avec le statut EN_ATTENTE, une référence unique (DEM-AAAA-NNN) est générée automatiquement, et une confirmation est retournée.
- En cas de pièces invalides (ok == false) : le système retourne demandeInvalide et l'utilisateur corrige son dossier.
- En cas d'authentification échouée : le flux est bloqué avec IdentificationEchouee.

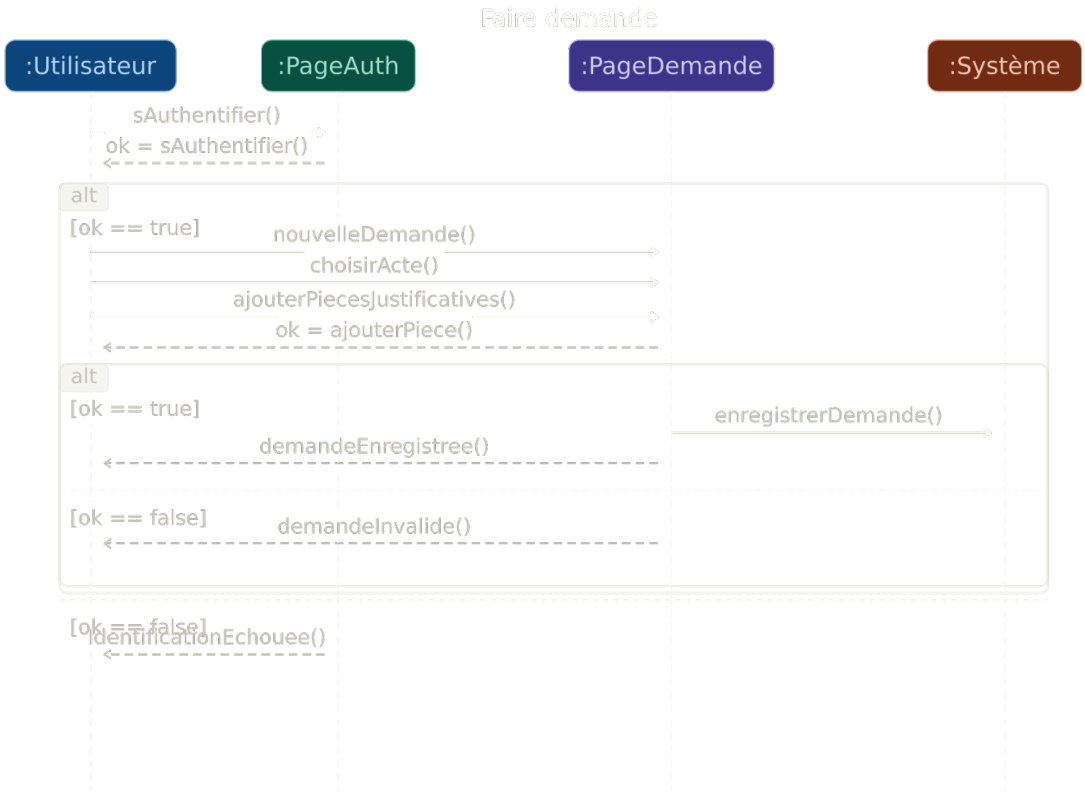

Figure 3 — Diagramme de séquence : Faire une demande

3.4 Traiter une demande (Agent)

> *Acteurs : AgentAdministratif. Précondition : l'agent est authentifié avec le rôle AGENT ou ADMIN.*

Ce diagramme décrit le traitement d'une demande par un agent administratif. Après authentification, l'agent consulte la liste des demandes en attente depuis la PageDemandeAdmin, sélectionne une demande et vérifie les pièces justificatives fournies.

- Si les pièces sont conformes (ok == true) : l'agent valide la demande, le système génère le document PDF correspondant (attestation, bulletin ou relevé), l'enregistre sur le système de fichiers et crée un ActeAdministratif en base de données lié à la demande.
- Si les pièces sont non conformes (ok == false) : l'agent annule la demande avec un motif de rejet (rejectReason).
- En cas d'échec d'authentification : AuthenticationEchouee est retourné.

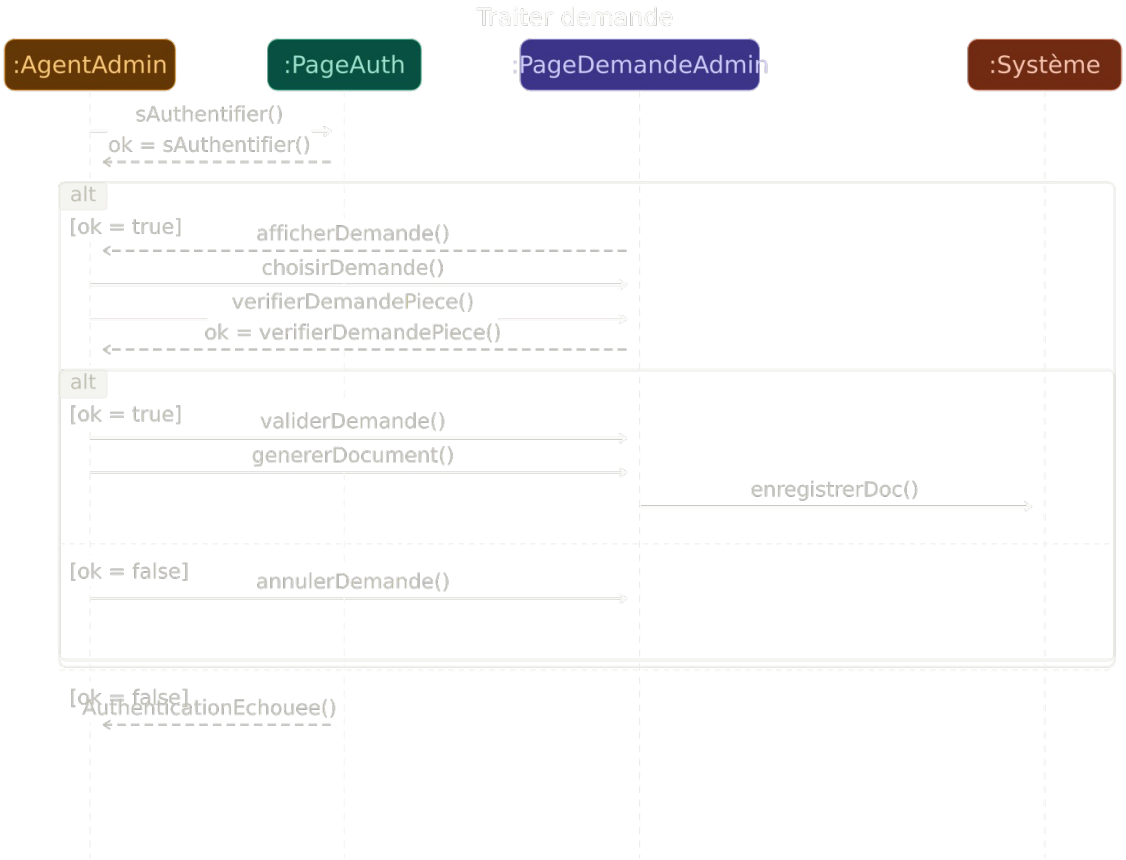

Figure 4 — Diagramme de séquence : Traiter une demande (Agent)

3.5 Générer un document (Usager)

> *Acteurs : Usager, AgentAdministratif. Ce diagramme présente la vue bout-en-bout du flux complet.*

Ce diagramme de séquence présente la vue d'ensemble du flux complet de génération d'un document, de la soumission par l'étudiant jusqu'au téléchargement du document final. Il fait intervenir quatre participants : l'Usager, la PageAuth, la PageDemande, l'AgentAdmin et le Système.

- L'usager s'authentifie, puis soumet sa demande avec ses informations (nom, prénom, pièce justificative, type de document).
- Le système enregistre la demande et notifie l'agent.
- L'agent valide les informations et déclenche la génération du document PDF.
- En cas de validation (ok == true) : le document est généré et l'usager peut le télécharger depuis son espace.
- En cas de rejet (ok == false) : l'usager reçoit une notification demandeRejetee avec le motif.

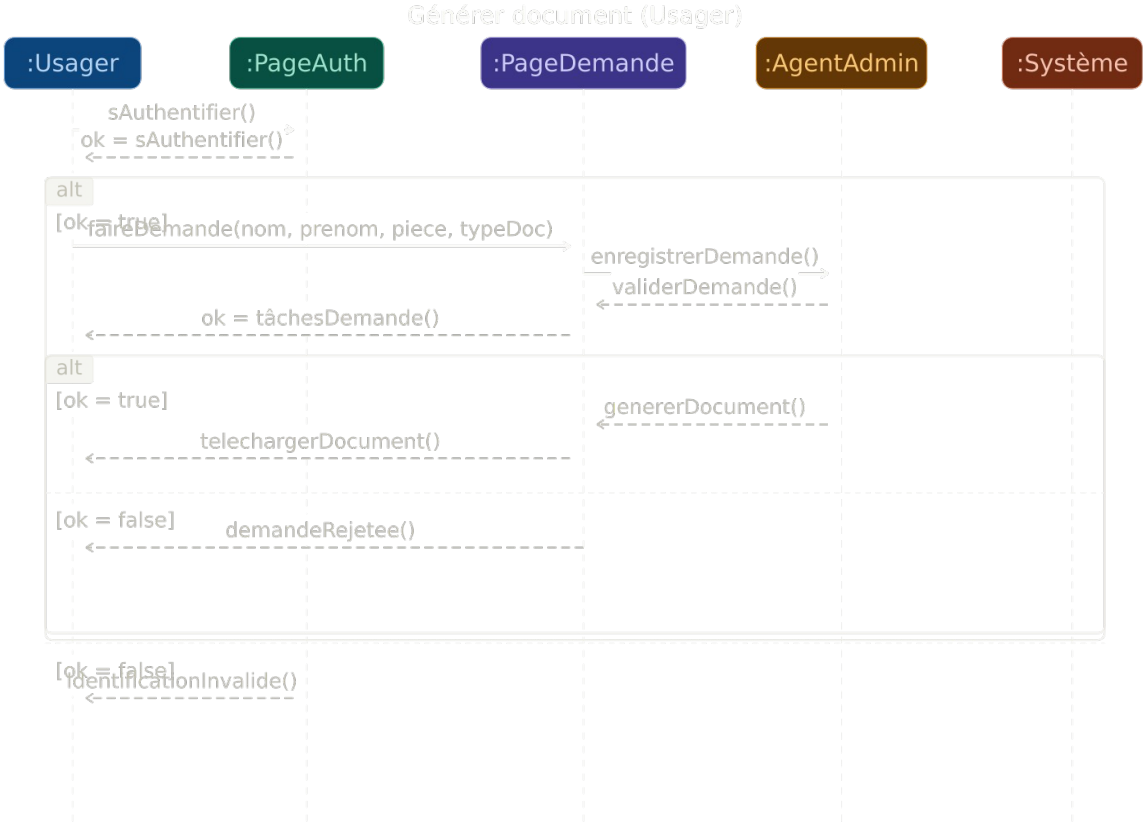

Figure 5 — Diagramme de séquence : Générer un document (Usager)

4. Diagramme d'Activités

> *Ce diagramme offre une vue transversale des flux de travail impliquant les trois acteurs principaux du système : Usager, Système et AgentAdmin.*

Le diagramme d'activités de CampusDocs représente l'ensemble du processus métier sous forme de flux d'activités réparties en trois couloirs (swim lanes) : l'Usager, le Système et l'AgentAdmin. Il permet de visualiser les responsabilités de chaque acteur à chaque étape du processus.

Flux principal

- L'Usager se connecte ou s'inscrit. Si le compte n'existe pas encore, il s'inscrit ; sinon, il se reconnecte.
- Une fois connecté et ses informations validées, l'Usager crée une nouvelle demande.
- Le Système enregistre la demande et notifie l'AgentAdmin.
- L'AgentAdmin analyse la demande : si les pièces sont conformes, il valide et génère le document ; sinon, il rejette avec un motif.
- En cas de validation, le Système génère le PDF et l'enregistre. L'Usager est notifié et peut télécharger son document.

Flux alternatifs

- Si la demande est incomplète ou refusée, l'Usager peut recommencer sa demande après correction.
- Si les informations de connexion sont invalides, l'Usager est invité à se reconnecter.
- L'Usager peut à tout moment annuler sa demande si elle est encore en statut EN_ATTENTE.

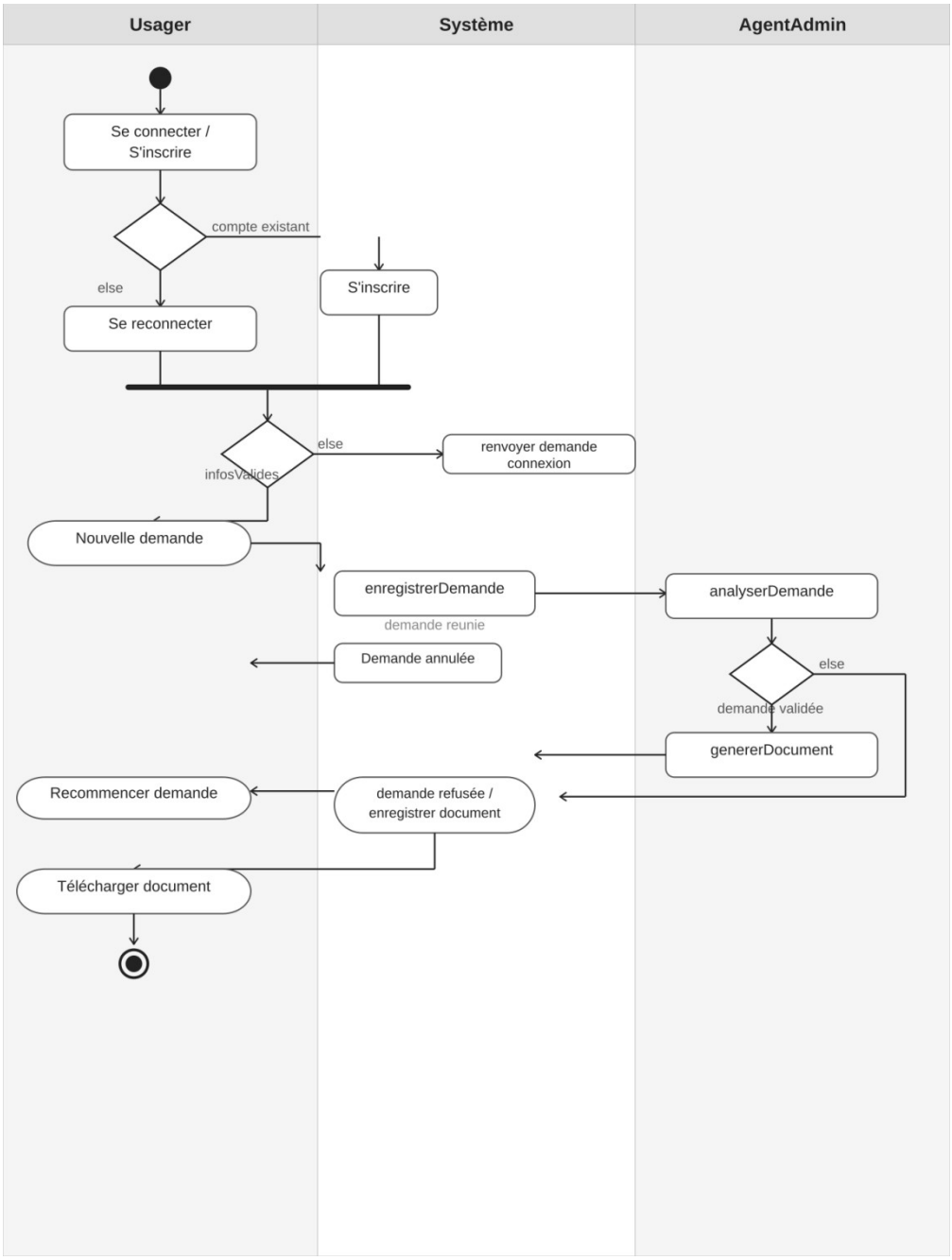

Figure 6 — Diagramme d'activités : Flux global CampusDocs

5. Diagramme de Composants

> *Le diagramme de composants décrit l'architecture logicielle statique du système, en montrant les modules, leurs interfaces et leurs dépendances.*

L'architecture de CampusDocs est organisée en trois sous-systèmes logiques distincts, communiquant via des interfaces bien définies :

5.1 Client JavaFX (Couche Présentation)

Le client est une application de bureau développée en JavaFX. Il est composé des éléments suivants :

- ApiClient : centralise tous les appels HTTP (GET, POST, PUT, DELETE, PATCH) vers l'API. Il gère l'injection automatique du token JWT dans chaque requête via le header Authorization.
- SessionManager : singleton qui maintient en mémoire les informations de la session courante (token JWT, rôle, nom, email, userId). Il est partagé entre tous les controllers JavaFX.
- AuthService (client) : orchestre le processus de login et stocke la réponse du serveur dans le SessionManager.
- Controllers (Login, Dashboard, Demande…) : gèrent l'affichage des vues FXML et les interactions utilisateur.
- Models (Demande, User…) : objets Java mappés depuis les réponses JSON via Gson.

5.2 API Spring Boot (Couche Métier)

Le serveur expose une API REST sécurisée sur le port 8080 :

- AuthController : gère le login, le signup et le logout. Génère et valide les tokens JWT.
- SecurityConfig + JwtFilter : applique les règles d'autorisation par rôle (ADMIN, AGENT, ETUDIANT) sur chaque endpoint. La politique est stateless (pas de session serveur).
- UserController / UserService : CRUD des utilisateurs et création des comptes agents avec hashage du mot de passe.
- DemandeController / DemandeService : gestion complète du cycle de vie des demandes (soumission, validation, rejet).
- ActeController / ActeService : consultation des actes générés et téléchargement des PDF.
- PdfService : génère les documents PDF (bulletins, attestations, relevés) et les persiste sur le système de fichiers.

5.3 Couche Données

Les repositories Spring Data JPA fournissent l'accès à la base de données MySQL :

- UserRepository, AgentRepository, UsagerRepository, AdminRepository : accès aux entités utilisateurs avec héritage JPA (stratégie JOINED).
- DemandeRepository : requêtes par userId, statut, et référence.
- ActeRepository : recherche des actes par demandeId, type et statut d'envoi.
- PieceRepository : gestion des pièces justificatives associées aux demandes.

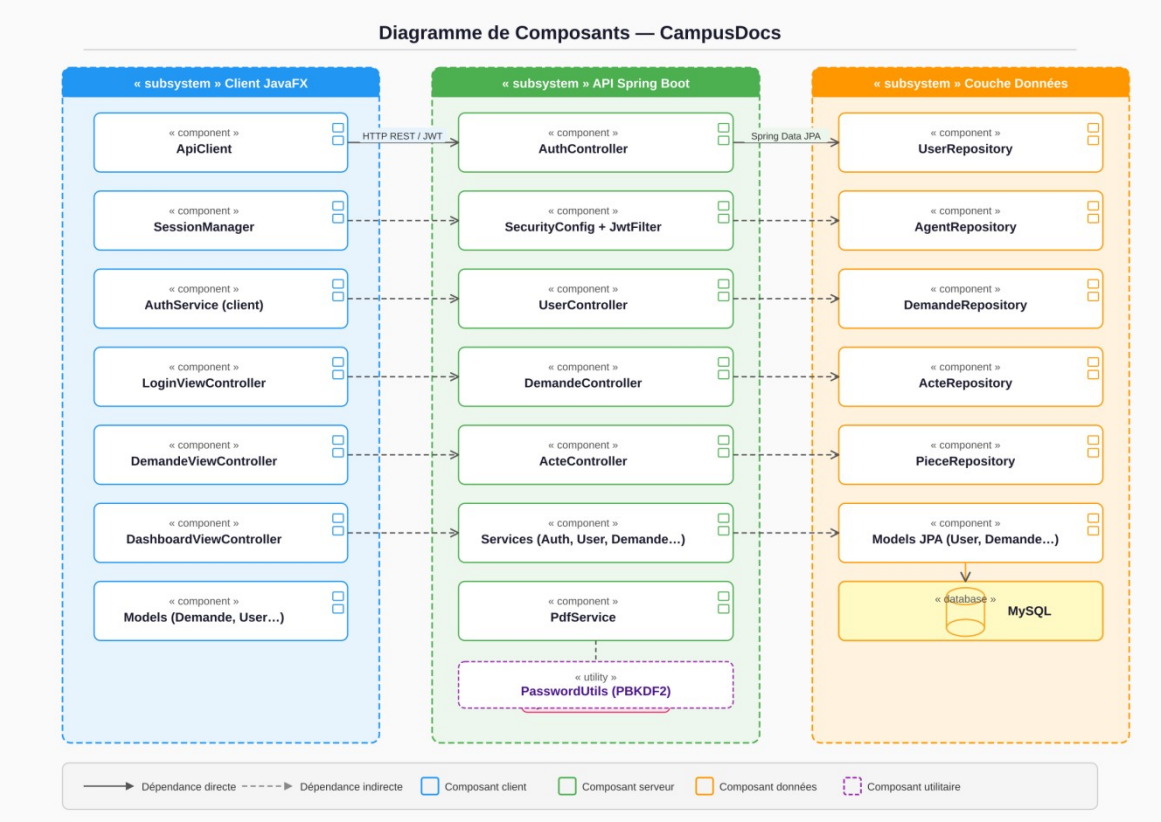

Figure 7 — Diagramme de composants : Architecture logicielle CampusDocs

6. Diagramme de Déploiement

> *Le diagramme de déploiement montre la répartition physique des artefacts logiciels sur les nœuds matériels et leur mode de communication.*

L'infrastructure de déploiement de CampusDocs comprend trois nœuds physiques distincts :

6.1 Poste Client

Le poste client est un ordinateur de bureau ou portable (Windows, Linux ou macOS) sur lequel est installée l'application JavaFX :

- CampusDocs.jar : le JAR exécutable contient l'intégralité de l'application client (vues FXML, controllers, services, modèles).
- JRE 17+ avec JavaFX Runtime : environnement d'exécution requis.
- config.properties : fichier de configuration local définissant l'URL de l'API (BASE_URL) et le mode de fonctionnement (DEV_MODE).
- SessionManager : maintient le token JWT en mémoire uniquement pendant la durée de la session. Aucune persistance locale du token.
- Vues FXML + CSS : définissent l'interface graphique de l'application.

6.2 Serveur Application

Le serveur applicatif tourne sous Ubuntu 24.04 LTS et héberge le backend Spring Boot :

- campusdocs-server.jar : le JAR Spring Boot embarque un serveur Tomcat, exposant l'API REST sur le port 8080.
- JRE 17 (OpenJDK) : environnement d'exécution Java avec Spring Security, JPA et Hibernate.
- JwtUtils : gestion des tokens JWT (algorithme HS256 via la bibliothèque JJWT).
- PasswordUtils : hashage des mots de passe avec PBKDF2/SHA-256 et sel aléatoire.
- PDF Storage (/storage/pdf/) : répertoire serveur contenant les documents PDF générés, organisés par type (attestations, bulletins, relevés).
- application.properties : configuration de la base de données, du port serveur et du secret JWT.
- SecurityConfig + JwtFilter : intercepte chaque requête pour valider le token et appliquer les contrôles d'accès RBAC.

6.3 Serveur Base de Données

Le serveur MySQL 8.x expose le service de base de données sur le port 3306 :

- campusdocs_db : schéma principal contenant toutes les tables du système.
- Table user : entité principale avec héritage JOINED vers agent_administratif, usager et administrateur.

- Table demande : enregistre toutes les demandes avec leur statut, référence, type et informations de traitement.
- Table acteAdministratif : stocke les actes générés, leur numéro de document et leur lien vers la demande d'origine.
- Table piece : pièces justificatives associées aux demandes (chemin du fichier, type, date d'upload).

6.4 Communications entre nœuds

Les nœuds communiquent via deux protocoles :

- Client ↔ Serveur Application : HTTP/1.1 REST sur le port 8080. Chaque requête porte un header Authorization: Bearer <JWT> pour l'authentification stateless.
- Serveur Application ↔ Base de données : JDBC/TCP-IP sur le port 3306. La connexion est gérée par le pool de connexions HikariCP intégré à Spring Boot.

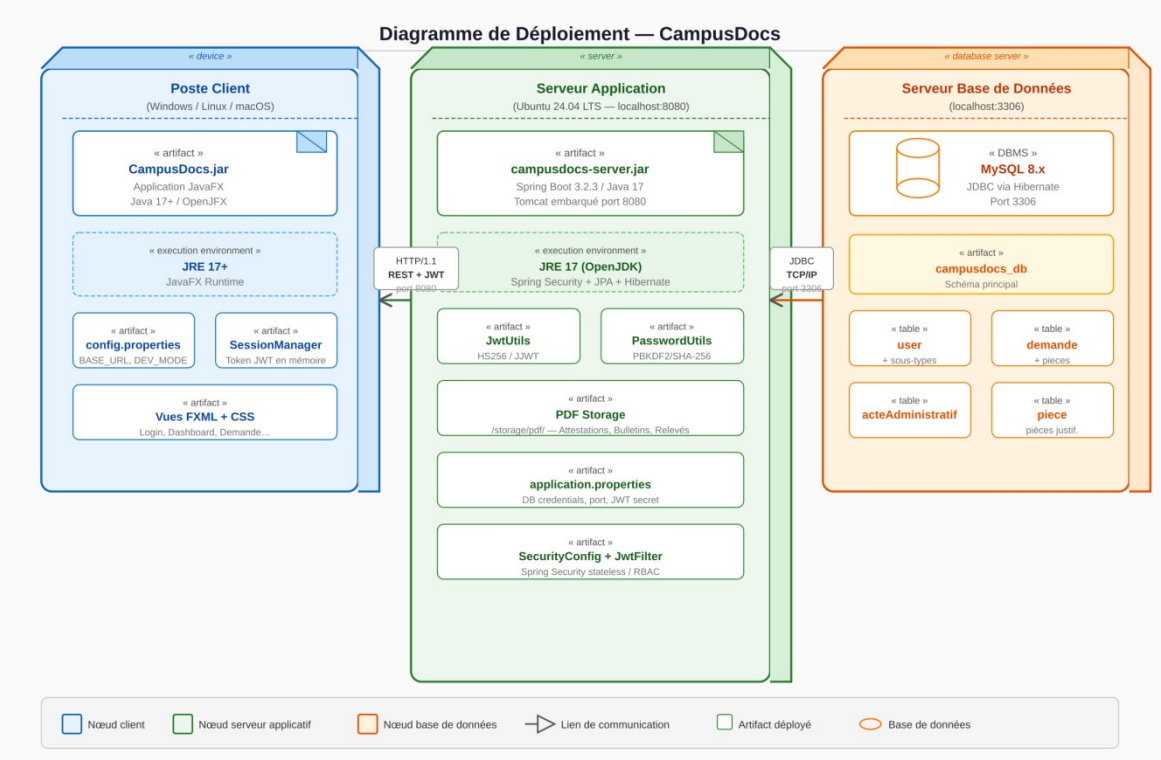

Figure 8 — Diagramme de déploiement : Infrastructure CampusDocs

7. Synthèse

L'ensemble des diagrammes UML présentés dans ce dossier couvre les trois dimensions fondamentales de la modélisation d'un système logiciel :

- Dimension comportementale (séquences + activités) : comment le système réagit aux actions des acteurs, dans quel ordre et avec quelles conditions alternatives.
- Dimension structurelle statique (composants) : comment les modules logiciels sont organisés, quelles interfaces ils exposent et de quels autres composants ils dépendent.
- Dimension structurelle physique (déploiement) : sur quels nœuds matériels les artefacts sont déployés et comment ces nœuds communiquent.

L'architecture retenue — client JavaFX / API Spring Boot / MySQL — garantit une séparation claire des responsabilités, une sécurité renforcée par JWT et hashage PBKDF2, et une extensibilité permettant d'ajouter de nouveaux types de documents ou de nouveaux rôles sans refactoring majeur.